

Java

Violet

2026-04-10

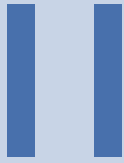
Chapter 0

目 录

II	JVM 标准库与常用工具
1.	集合框架 6
1.1.	集合类概述 6
1.2.	List 接口与实现 6
1.3.	Set 接口与实现 8
1.4.	Map 接口与实现 9
1.5.	ConcurrentHashMap 原理 11
1.6.	集合遍历与 fail-fast 11
1.7.	多线程集合类 12
1.8.	深拷贝 12
1.9.	补充说明 13
1.10.	总结 15
2.	异常处理 16
2.1.	异常概述 16
2.2.	异常类型与层次结构 17
2.3.	异常处理机制 18
2.4.	自定义异常 21
2.5.	最佳实践与反模式 21
2.6.	面试与实战高频 23



Java & Kotlin 核心基础



JVM 标准库与常用工具

1. 集合框架	6
2. 异常处理	16

Chapter 1

集合框架

1 集合类概述

Java 集合框架用于统一管理数据集合，核心目标是：统一操作接口、屏蔽底层实现差异、提供可组合的遍历与算法能力。

INFO

图示占位：这里应有“Java 集合框架概览图（Collection / List / Set / Map 及常见实现类）”。

1.1 分类与定位

- 单列集合：以 `Collection` 为根（如 `List`、`Set`）
- 双列集合：以 `Map` 为根（键值对结构）

1.2 常见实现类速览

实现类	底层结构	核心特点	典型场景
<code>ArrayList</code>	动态数组	随机访问快，尾插高效	读多写少
<code>LinkedList</code>	双向链表	中间插删更友好	频繁插删
<code>HashSet</code>	哈希表	去重快、通常无序	成员判重
<code>TreeSet</code>	红黑树	自动排序	有序去重
<code>HashMap</code>	哈希表	查询快	通用键值存储
<code>TreeMap</code>	红黑树	按键有序	范围查询

NOTE

`List` 与 `Set` 的关键区别：

- `List`：有序、可重复、可按索引访问
- `Set`：元素唯一、通常不按索引访问

2 List 接口与实现

2.1 List 接口特点

`List` 是有序集合，允许重复元素，支持按索引访问。

方法	作用	备注
<code>add(E e)</code>	末尾添加元素	可重复
<code>get(int index)</code>	按索引读取	随机访问
<code>set(int index, E e)</code>	替换元素	返回旧值
<code>remove(int index)</code>	按索引删除	后续元素前移
<code>subList(int from, int to)</code>	截取子列表	左闭右开

Collection 基础方法（父接口）

方法	描述	注意点
<code>add</code>	添加元素	返回是否成功
<code>remove</code>	删除匹配元素	通常删首个匹配
<code>contains</code>	是否包含元素	依赖 <code>equals</code>
<code>size</code>	元素个数	
<code>iterator</code>	获取迭代器	遍历入口
<code>toArray</code>	转数组	

2.2 ArrayList

`ArrayList` 底层是动态数组，特点是读取快、扩容有成本。

核心特征

1. 随机访问 `O(1)`
2. 末尾追加均摊 `O(1)`
3. 中间插删通常 `O(n)`（涉及元素搬移）

```
1 List<String> names = new ArrayList<>();
2 names.add("Alice");
3 names.add("Bob");
4 names.add(1, "Tom");
5 System.out.println(names.get(0));
6 System.out.println(names);
```



2.3 LinkedList

`LinkedList` 底层是双向链表，支持头尾快速操作，也实现了 `Deque`。

核心特征

1. 头尾插删效率高
2. 中间插删更友好（定位后修改链接）
3. 按索引随机访问慢（需要遍历）

```
1 LinkedList<Integer> queue = new LinkedList<>();
2 queue.addLast(10);
3 queue.addLast(20);
4 queue.addFirst(5);
5 System.out.println(queue.removeFirst());
6 System.out.println(queue);
```



2.4 Vector 与 Stack

`Vector` 是早期线程安全动态数组实现，整体性能通常不如 `ArrayList`。`Stack` 基于 `Vector`，遵循 LIFO。

TIP

新代码中栈结构优先考虑 `ArrayDeque`，通常更轻量、性能更好。

2.5 ArrayList 与 LinkedList 对比

对比项	ArrayList	LinkedList
底层结构	动态数组	双向链表
随机访问	快 ($O(1)$)	慢 ($O(n)$)
中间插删	慢 (搬移元素)	相对更合适
内存特性	连续空间，缓存友好	节点额外指针开销
典型场景	读多写少	频繁插删

3 Set 接口与实现

`Set` 的核心语义是“不重复”。重复判定依赖 `equals()` 与 `hashCode()`（哈希实现）或比较器（有序树实现）。

3.1 HashSet

基于哈希表，通常无序，查找/插入/删除平均接近 $O(1)$ 。

3.2 LinkedHashSet

在哈希结构上维护插入顺序，适合“去重 + 保序”。

3.3 TreeSet

基于红黑树，自动排序，复杂度通常 $O(\log n)$ 。

TreeSet 常用有序方法

方法	作用
first() / last()	取首尾元素
pollFirst() / pollLast()	取并删首尾元素
headSet(to)	小于 to 的子集
subSet(from, to)	区间子集
tailSet(from)	大于等于 from 的子集

3.4 Set 实现对比

实现	有序性	复杂度	null	场景
HashSet	通常无序	平均 $O(1)$	支持	快速去重
LinkedHashSet	按插入顺序	平均 $O(1)$	支持	去重并保序
TreeSet	按比较规则	$O(\log n)$	默认不支持 (依赖比较器)	有序集合

4 Map 接口与实现

Map 保存键值映射，键唯一，值可重复。

4.1 Map 常用方法

方法	作用
put	新增/覆盖键值
get	按 key 查 value
remove	删除映射
containsKey	是否存在 key
entrySet	遍历键值对

4.2 HashMap

HashMap 是最常用实现，JDK 8 起底层可概括为“数组 + 链表 + 红黑树”。

关键参数

参数	默认值	说明
initialCapacity	16	初始桶数量 (2 的幂)
loadFactor	0.75	触发扩容阈值比例

TREEIFY_THRESHOLD	8	链表树化阈值
UNTREEIFY_THRESHOLD	6	树退化阈值
MIN_TREEIFY_CAPACITY	64	最小树化容量

put 流程（简化）

1. 计算 key 哈希并定位桶
2. 桶为空则直接插入
3. 桶非空则处理冲突（链表/树）
4. 超阈值触发扩容或树化

```
1 Map<String, Integer> scores = new HashMap<>();
2 scores.put("Java", 95);
3 scores.put("Python", 92);
4 scores.put(null, 100);
5 System.out.println(scores.get("Java"));
```

Java

WARNING

作为 key 的自定义对象，必须正确重写 `equals()` 与 `hashCode()`，否则可能出现“放得进去、取不出来”。

图示占位：HashMap 底层结构

INFO

图示占位：这里应有“HashMap 数组-桶-链表-红黑树结构图”。

4.3 LinkedHashMap

在 HashMap 基础上维护双向链表，支持插入顺序或访问顺序。

```
1 Map<Integer, String> cache = new LinkedHashMap<>(4, 0.75f, true) {
2     @Override
3     protected boolean removeEldestEntry(Map.Entry<Integer, String> eldest) {
4         return size() > 3;
5     }
6 };
```

Java

图示占位：LRU 访问顺序

INFO

图示占位：这里应有“LinkedHashMap(accessOrder=true) 的 LRU 淘汰示意图”。

4.4 TreeMap

基于红黑树，按 key 有序，范围查询友好，复杂度通常 $O(\log n)$ 。

4.5 HashMap 1.7 vs 1.8（面试高频）

维度	JDK 1.7	JDK 1.8
结构	数组+链表	数组+链表+红黑树
插入	头插法	尾插法
扩容迁移	重新计算更重	迁移路径优化
并发风险	可能出现环链死循环	修复环链问题但仍非线程安全

5 ConcurrentHashMap 原理

5.1 为什么 HashMap 不能直接并发用

1. 并发写可能数据覆盖
2. 历史版本扩容存在环链风险
3. 复合操作（先判再改）非原子

5.2 JDK 1.7 与 1.8 的并发实现差异

版本	核心机制	锁粒度	特点
JDK 1.7	Segment 分段锁	段级	并发度受段数影响
JDK 1.8	CAS + synchronized	桶级	并发度更高

5.3 选型建议

- 高并发键值存储优先 `ConcurrentHashMap`
- 读多写少列表可考虑 `CopyOnWriteArrayList`

6 集合遍历与 fail-fast

6.1 Iterator / ListIterator / Spliterator

接口	能力	典型场景
Iterator	单向遍历 + 删除当前	通用遍历
ListIterator	双向遍历 + 就地增删改	List 增强遍历
Spliterator	可拆分遍历	Stream 并行支撑

6.2 fail-fast 机制

普通集合迭代器通常会在检测到结构性并发修改时抛 `ConcurrentModificationException`。

```
1 List<Integer> list = new ArrayList<>(List.of(1, 2, 3));
2 for (Integer x : list) {
3     if (x == 2) {
```



```
4     list.remove(x); // 可能触发 ConcurrentModificationException
5 }
6 }
```

NOTE

fail-fast 是“尽早暴露误用”的机制，不等价于线程安全保障。

6.3 安全修改建议

1. 遍历中删除优先 `Iterator.remove()`
2. 条件删除优先 `removeIf()`
3. 并发场景改用并发容器

7 多线程集合类

7.1 并发集合速查

集合	特点	适用场景
ConcurrentHashMap	高并发键值读写	缓存、路由表
CopyOnWriteArrayList	读无锁，写复制	读多写少配置
ConcurrentSkipListMap	并发有序映射	有序并发访问
LinkedBlockingQueue	阻塞 FIFO	生产者-消费者
ArrayBlockingQueue	有界阻塞队列	限流缓冲

7.2 普通集合 vs 并发集合

场景	推荐
单线程	ArrayList / HashMap
高并发读写	ConcurrentHashMap
读多写少	CopyOnWriteArrayList
任务队列	BlockingQueue 系列

8 深拷贝

8.1 深拷贝与浅拷贝

- 浅拷贝：复制外层对象，内部引用共享
- 深拷贝：递归复制内部引用对象

8.2 常见实现方式

构造函数复制

优点是可控、类型安全；缺点是对象层级深时代码冗长。

clone() 重写

需实现 `Cloneable`，并在引用字段上继续深拷贝。

序列化/反序列化复制

可借助 Commons Lang、Gson、Jackson 等，开发效率高，但需关注性能与类型约束。

方式	优点	缺点	适用
构造函数	显式可控	模板代码多	小中型对象
clone	JDK 原生	易误用浅拷贝	中等复杂度
序列化	实现快	性能开销较大	工具化场景

9 补充说明

9.1 正确重写 equals 与 hashCode

放入哈希集合（`HashSet` / `HashMap`）的自定义对象，必须满足：

- 1. `equals` 相等的对象，`hashCode` 必须相等
- 2. 同一次运行期间，不变字段对应的 `hashCode` 应保持稳定

```
1 public class Person {
2     private String name;
3     private int age;
4
5     @Override
6     public boolean equals(Object o) {
7         if (this == o) return true;
8         if (o == null || getClass() != o.getClass()) return false;
9         Person p = (Person) o;
10        return age == p.age && Objects.equals(name, p.name);
11    }
12
13    @Override
14    public int hashCode() {
15        return Objects.hash(name, age);
16    }
17 }
```

9.2 集合选型经验法则

需求	首选
按索引随机访问	ArrayList

频繁头尾操作	ArrayDeque / LinkedList
快速去重	HashSet
去重并保序	LinkedHashSet
有序键值查询	TreeMap
高并发键值	ConcurrentHashMap

9.3 时间复杂度参考

操作	ArrayList	LinkedList	HashMap	TreeMap
添加	均摊 O(1)	O(1)	平均 O(1)	O(log n)
删除	O(n)	按索引/值通常 O(n)	平均 O(1)	O(log n)
查找	O(1) 按索引	O(n)	平均 O(1)	O(log n)

9.4 不可变集合与只读视图

很多同学会把“不可变集合”和“只读视图”混为一谈，二者并不等价：

1. `List.of()` / `Set.of()` / `Map.of()` 返回真正不可变集合
2. `Collections.unmodifiableList(x)` 返回的是“只读视图”，底层 `x` 变了，视图也会变

```

1 List<String> src = new ArrayList<>(List.of("A", "B"));
2 List<String> view = Collections.unmodifiableList(src);
3 src.add("C");
4 System.out.println(view); // [A, B, C]
5
6 List<String> imm = List.of("X", "Y");
7 // imm.add("Z"); // UnsupportedOperationException

```

WARNING

对外暴露集合时，若要求“状态绝对不可被修改”，优先返回拷贝后的不可变集合，而不是只读视图。

9.5 Map 遍历性能建议

遍历 `Map` 时通常优先 `entrySet()`，避免“先遍历 key 再 `get(value)`”的重复查找开销。

```

1 // 推荐
2 for (Map.Entry<String, Integer> e : map.entrySet()) {
3     String k = e.getKey();
4     Integer v = e.getValue();
5 }
6
7 // 次优（会重复哈希查找）
8 for (String k : map.keySet()) {
9     Integer v = map.get(k);
10 }

```

9.6 常见易错点清单

1. 在 `for-each` 中直接调用集合自身 `remove()`，触发 `ConcurrentModificationException`
2. 自定义对象做 `HashMap` key 却未同时重写 `equals/hashCode`
3. 误把 `LinkedList` 当作高性能随机访问容器
4. 并发场景继续使用 `HashMap` / `ArrayList` 做共享可变状态
5. 误以为 `unmodifiable*` 返回的是“深层不可变对象”

10 总结

掌握集合框架的关键不是背类名，而是理解这几个维度：

1. 底层结构（数组/链表/哈希/树/跳表）
2. 复杂度特征（时间与空间）
3. 语义特征（有序、可重复、null 支持）
4. 并发特征（线程安全与迭代一致性）

一句话：先看访问模式，再选数据结构，最后看并发与可维护性。

Chapter 2

异常处理

1 异常概述

Java 中的异常处理机制与 C# 的核心思想类似，都是通过 `try-catch-finally` 捕获和处理运行问题。但 Java 在类型体系上更严格，尤其是“受检异常”会被编译器强制检查。

1.1 基本语法结构

```
1 try {  
2     // 需要进行异常捕获的代码块  
3 } catch (Exception ex) {  
4     // 捕获异常信息；可定义多个 catch，分别处理不同异常  
5 } finally {  
6     // 无论是否发生异常，都会尝试执行  
7 }
```

1.2 异常体系速览

异常根类型是 `java.lang.Throwable`，其下分为两条主线：

- `Error`：系统级严重错误，通常不建议业务代码处理
- `Exception`：程序可处理异常

`Exception` 下又分为：

- `RuntimeException` 及其子类：非检查性异常（unchecked）
- 其他 `Exception` 子类：检查性异常（checked）

NOTE

Java 的核心设计是：可预期且可恢复的问题，尽量在编译期逼迫开发者显式处理。

1.3 常见异常一览

非检查性异常（`RuntimeException`）

异常	描述
<code>ArithmeticException</code>	整数除零等算术非法操作
<code>ArrayIndexOutOfBoundsException</code>	数组下标越界

ClassCastException	类型强转失败
IllegalArgumentException	传入参数不合法
IllegalStateException	对象状态不符合当前操作要求
NegativeArraySizeException	创建了负长度数组
NullPointerException	在需要对象处使用了 null
NumberFormatException	字符串无法转换为数值
UnsupportedOperationException	调用了不支持的操作

检查性异常 (Checked Exception)

异常	描述
ClassNotFoundException	动态加载类时未找到目标类
CloneNotSupportedException	对象未实现 Cloneable 却调用 clone
IllegalAccessException	访问权限不满足
InstantiationException	试图实例化抽象类/接口
InterruptedException	线程被中断
NoSuchFieldException	反射访问字段不存在
NoSuchMethodException	反射访问方法不存在

1.4 Throwable 常用方法

方法	说明
getMessage()	返回异常简要描述信息
getCause()	返回导致当前异常的根因异常
toString()	返回异常类型 + 消息
printStackTrace()	打印完整堆栈
getStackTrace()	返回堆栈数组供程序分析
fillInStackTrace()	填充当前堆栈信息

2 异常类型与层次结构

2.1 Throwable / Error / Exception

`Throwable` 是“可被抛出对象”的统一抽象。只有 `Throwable` 或其子类实例，才可以被 `throw` 抛出。

系统错误 (Error)

Error 一般由 JVM 抛出，表示严重运行问题。常见如：

类	可能原因
LinkageError	类依赖在编译后发生不兼容变更
VirtualMachineError	JVM 运行资源不足或内部错误
OutOfMemoryError	内存耗尽
NoClassDefFoundError	类加载失败
StackOverflowError	递归过深导致栈溢出

WARNING

Error 通常不属于可恢复业务异常。生产系统应以“告警 + 降级 + 保护性终止”为主，而不是继续执行业务逻辑。

运行时异常 (RuntimeException)

运行时异常通常反映程序逻辑错误，编译器不会强制捕获或声明。

- 常见场景：空引用访问、参数不合法、数组越界、类型转换失败
- 处理策略：优先修复根因，必要时在边界层统一兜底

编译时异常 (Checked Exception)

除 **RuntimeException** 及其子类外，大多数 **Exception** 都是受检异常，必须显式处理：

1. 使用 **try-catch** 捕获
2. 或在方法签名中用 **throws** 继续上抛

TIP

“可恢复并可预期”的问题，适合用 checked exception；“编程错误”通常属于 unchecked exception。

3 异常处理机制

3.1 声明异常 (throws)

方法可在签名中声明可能抛出的受检异常：

```
1 public void readFile() throws IOException {  
2     // ...  
3 }  
4  
5 public void process() throws IOException, SQLException {  
6     // ...  
7 }
```

重写方法时的 throws 规则

1. 子类重写方法抛出的受检异常，不能比父类更宽
2. 可以抛出父类已声明异常的子类
3. 父类方法未声明受检异常时，子类不能新增受检异常声明

3.2 抛出异常 (throw)

在检测到非法状态时主动抛出异常：

```
1 if (age < 0) {  
2     throw new IllegalArgumentException("年龄不能为负数");  
3 }
```

 Java

throw 与 throws 区别

- `throw`：抛出一个具体异常对象
- `throws`：声明方法可能抛出的异常类型

3.3 捕获异常 (try-catch-finally)

推荐按“最具体异常到最通用异常”的顺序编写多重 catch：

```
1 try {  
2     String s = args[0];  
3     int n = Integer.parseInt(s);  
4     int r = 10 / n;  
5 } catch (ArrayIndexOutOfBoundsException e) {  
6     System.out.println("缺少参数");  
7 } catch (NumberFormatException e) {  
8     System.out.println("参数必须是数字");  
9 } catch (ArithmeticException e) {  
10    System.out.println("除数不能为零");  
11 } catch (Exception e) {  
12    System.out.println("未知错误: " + e.getMessage());  
13 } finally {  
14    System.out.println("资源收尾");  
15 }
```

 Java

finally 执行时机

1. try 正常结束后执行 finally
2. try 抛异常并被 catch 处理后执行 finally
3. try 抛异常未被当前方法处理，也会先执行 finally 再向上抛
4. try/catch 中出现 return，finally 仍会在方法返回前执行

return 与 finally 的交互

```
1 public static int test1() {  
2     try {  
3         return 1;  
4     }
```

 Java

```
4    } finally {  
5        System.out.println("finally");  
6    }  
7 }  
8  
9 public static int test2() {  
10     try {  
11         return 1;  
12     } finally {  
13         return 2; // 会覆盖 try 中的返回值（不推荐）  
14     }  
15 }
```

DANGER

不建议在 finally 中写 return。它会覆盖原返回值，甚至吞掉原始异常，导致排障困难。

finally 可能不执行的极端情况

- 调用 `System.exit()` 直接终止 JVM
- JVM 崩溃或严重错误导致进程中止
- 线程被强制终止（历史 API，如 `Thread.stop()`）

3.4 try-with-resources (Java 7+)

对于实现 `AutoCloseable` 的资源，优先使用自动关闭语法：

```
1 try (BufferedReader br = new BufferedReader(new FileReader("file.txt"))) {  
2     String line = br.readLine();  
3     System.out.println(line);  
4 } catch (IOException e) {  
5     e.printStackTrace();  
6 }
```

 Java

suppressed 异常说明（易忽略）

当 `try` 块和资源关闭过程都抛异常时：

1. 主异常通常是 `try` 块中的异常
2. 关闭资源时产生的异常会被放入 `getSuppressed()`

```
1 try (MyResource r = new MyResource()) {  
2     throw new RuntimeException("主异常");  
3 } catch (Exception e) {  
4     System.out.println(e.getMessage()); // 主异常  
5     for (Throwable s : e.getSuppressed()) {  
6         System.out.println("suppressed: " + s.getMessage());  
7     }  
8 }
```

 Java**NOTE**

排查线上问题时，不要只看 `getMessage()`；应同时检查 `getCause()` 与 `getSuppressed()`，否则可能漏掉资源关闭阶段的关键异常。

4 自定义异常

4.1 何时使用自定义异常

当内置异常无法准确表达业务语义时，应定义业务异常类。例如：订单状态非法、余额不足、重复提交。

4.2 自定义异常示例

```
1 public class MyException extends Exception {
2     public MyException(String message) {
3         super(message);
4     }
5 }
6
7 try {
8     int a = -1;
9     if (a < 0) throw new MyException("a 不能小于 0");
10    int[] array = new int[a];
11 } catch (MyException e) {
12     System.out.println(e.getMessage());
13 } finally {
14     System.out.println("异常捕获结束");
15 }
```

命名与设计建议

1. 异常类名统一以 `Exception` 结尾
2. 需强制调用方处理时继承 `Exception`
3. 业务运行时错误可继承 `RuntimeException`
4. 保留异常链，优先使用带 `cause` 的构造方法

5 最佳实践与反模式

5.1 推荐实践

1. 只捕获你能处理的异常
2. 不要吞异常；至少记录日志
3. 保留异常链，不要丢失原始堆栈
4. 优先用具体异常类型，而不是泛化到 `Exception`
5. 资源释放优先使用 `try-with-resources`

`InterruptedException` 处理规范

对于 `InterruptedException`，常见正确姿势是“恢复中断标记”或“继续向上抛出”：

```
1 try {
```

```
2 Thread.sleep(1000);
3 } catch (InterruptedException e) {
4     Thread.currentThread().interrupt(); // 恢复中断标记
5     throw new RuntimeException("线程被中断", e);
6 }
```

WARNING

不要简单吞掉 `InterruptedException`。这会破坏线程协作协议，导致线程池任务无法按预期停止。

异常日志建议

记录异常时应包含：

1. 业务上下文（请求 ID、用户 ID、关键参数）
2. 异常类型 + 异常消息 + 完整堆栈
3. 关键分支状态（重试次数、远端响应码）

并避免：

- 只打印 `e.getMessage()` 不打印堆栈
- 重复多层打印同一异常造成日志噪声
- 将密码、令牌等敏感信息写入日志

分层异常转换（Service / Controller）

推荐在分层架构中做“边界转换”：

1. DAO 层抛技术异常（SQL/IO）
2. Service 层转换为业务异常并补充语义
3. Controller 层统一映射为错误码与对外响应

```
1 try {
2     userRepository.save(user);
3 } catch (SQLException e) {
4     throw new UserOperationException("用户保存失败", e);
5 }
```

 Java**TIP**

统一异常映射（例如 Web 全局异常处理器）能显著提升接口一致性，减少散落在控制器中的重复 `try-catch`。

5.2 反模式示例

```
1 // 反模式：吞异常
2 try {
3     doSomething();
4 } catch (Exception e) {
5     // nothing
6 }
7
8 // 反模式：抛新异常但丢失原始堆栈
9 try {
10    doSomething();
```

 Java

```
11 } catch (IOException e) {  
12     throw new RuntimeException(e.getMessage());  
13 }  
14  
15 // 正确: 保留 cause  
16 try {  
17     doSomething();  
18 } catch (IOException e) {  
19     throw new RuntimeException("处理失败", e);  
20 }
```

6 面试与实战高频

6.1 Checked vs Unchecked

维度	Checked Exception	Unchecked Exception
编译器检查	强制检查	不强制检查
处理要求	必须捕获或声明	可自行选择
典型类型	IOException、SQLException	NullPointerException、IllegalArgumentException
设计意图	可恢复、可预期问题	编程错误或非法状态

6.2 高频问答

try 里 return, finally 会执行吗

会执行。finally 在方法真正返回前执行。

try/catch/finally 都有 return, 最终返回哪个

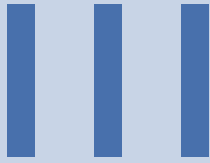
finally 中的 return 会覆盖前面的返回值。

finally 中修改变量为何有时生效有时不生效

- 基本类型: return 时值已拷贝, finally 修改通常不影响返回
- 引用类型: return 的是引用, finally 修改对象内容通常可见

TIP

生产代码里尽量避免在 finally 中修改返回路径。可读性差、风险高、容易引入隐蔽 bug。



高级特性与函数式编程

IV

并发编程



网络编程与高性能 IO



JVM 底层原理与性能调优